



COMILLAS

UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Tema 4 (continuación): Fórmulas intermedias entre MC y TD a un paso

Predicción: Monte Carlo y TD a un paso

- Cómo actualizamos la predicción con métodos de **Monte Carlo**:

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)],$$

- Ahora, siguiendo la idea de bootstrapping, una regla simple de actualización de **TD** sería:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- El objetivo en el caso TD trae todo el futuro a un paso.

¿Podríamos hacer
bootstrapping n
pasos después?

1-step TD
and TD(0)



2-step TD



3-step TD



...

n-step TD



...

∞ -step TD
and Monte Carlo



El retorno a n pasos

- Podemos definir de manera natural el retorno a n pasos como:

$$G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n})$$

- Con él, generalizamos el aprendizaje TD, teniendo un espectro de opciones entre el TD a un paso y MC:

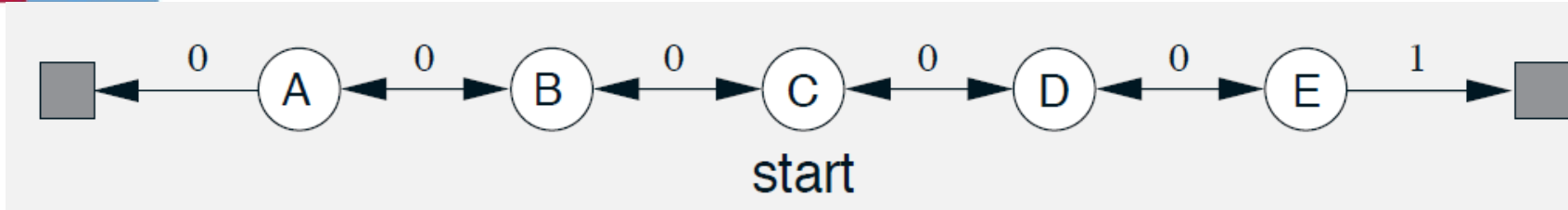
$$V_{t+n}(S_t) \doteq V_{t+n-1}(S_t) + \alpha [G_{t:t+n} - V_{t+n-1}(S_t)]$$

- Supon que tomamos $n=10$. ¿Qué pasa al principio de un episodio?

¿Hay un valor óptimo de n ?

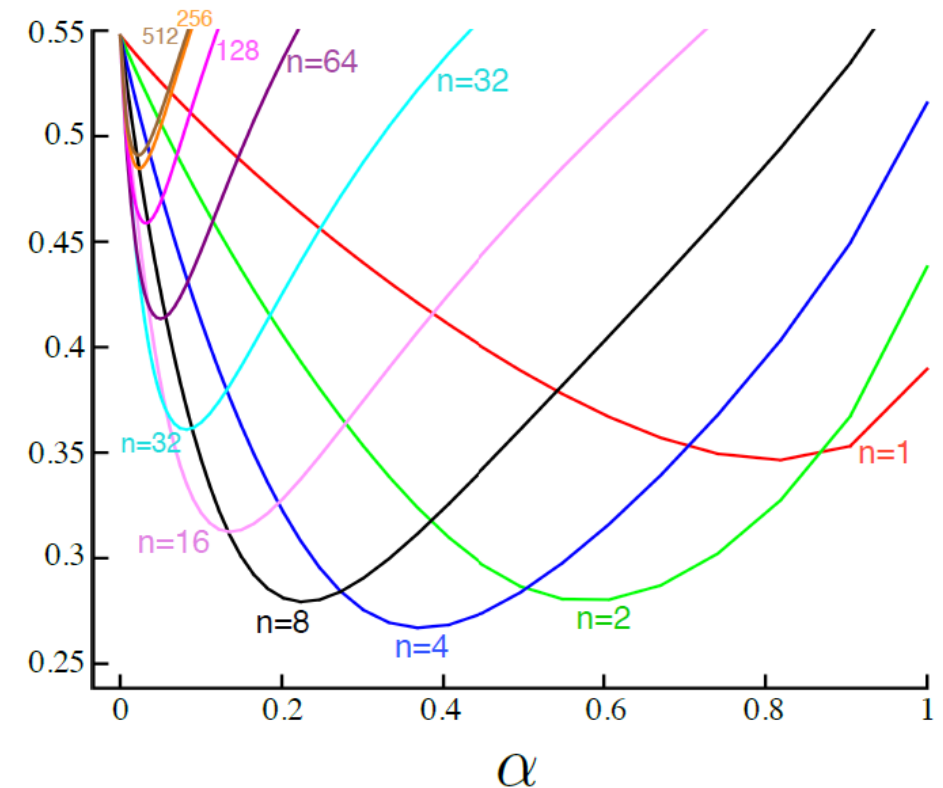
- n aparece como un hiperparámetro del método de aprendizaje.
- Valores pequeños llevarán a estimadores con más sesgo, pero menos varianza.
- También habrá implicaciones en cuántos estados se verán afectados por el efecto de una muestra de experiencia

Ejemplo random walk



- Para un caso con más estados, observamos los siguientes errores tras 10 episodios

Average
RMS error
over 19 states
and first 10
episodes





Sarsa a n pasos

Loop for each episode:

Initialize and store $S_0 \neq \text{terminal}$

Select and store an action $A_0 \sim \pi(\cdot|S_0)$

$T \leftarrow \infty$

Loop for $t = 0, 1, 2, \dots$:

| If $t < T$, then:

| Take action A_t

| Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

| If S_{t+1} is terminal, then:

| $T \leftarrow t + 1$

| else:

| Select and store an action $A_{t+1} \sim \pi(\cdot|S_{t+1})$

| $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)

| If $\tau \geq 0$:

| $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

| If $\tau + n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)

| $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha [G - Q(S_\tau, A_\tau)]$

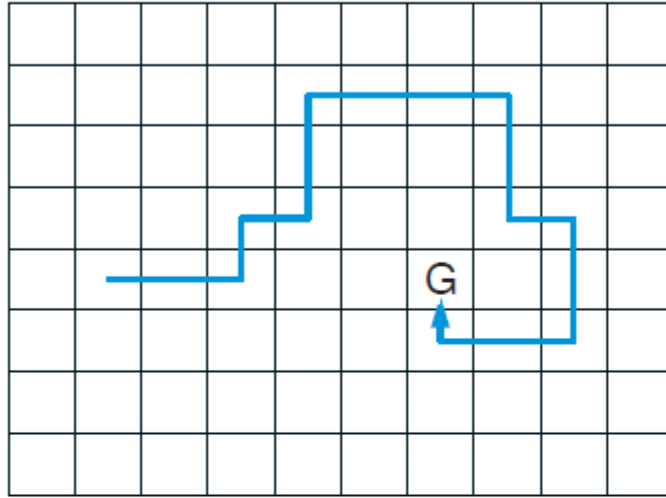
| If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is ε -greedy wrt Q

Until $\tau = T - 1$

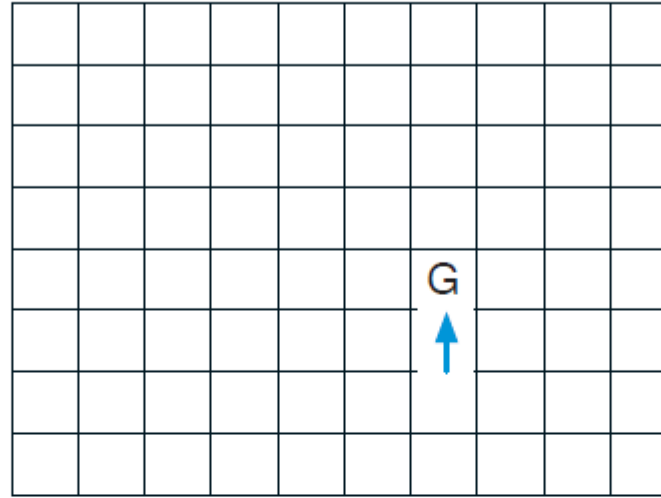


Sarsa a n pasos

Path taken



Action values increased
by one-step Sarsa



Action values increased
by 10-step Sarsa

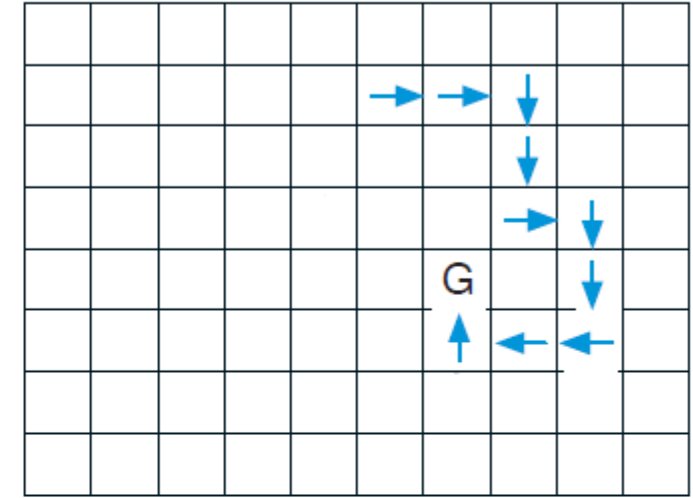


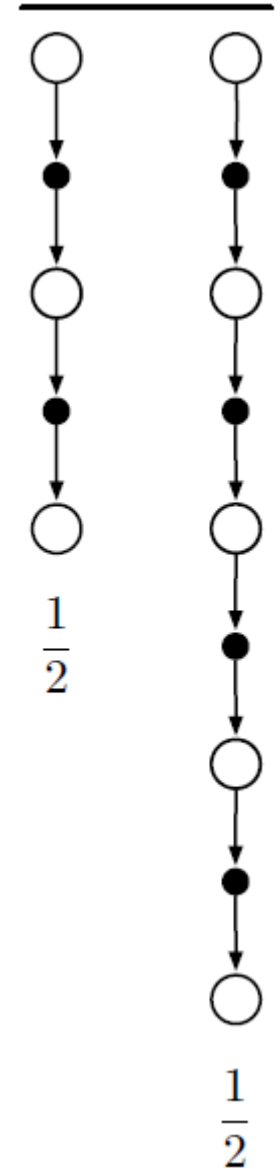
Figure 7.4: Gridworld example of the speedup of policy learning due to the use of n -step methods. The first panel shows the path taken by an agent in a single episode, ending at a location of high reward, marked by the G. In this example the values were all initially 0, and all rewards were zero except for a positive reward at G. The arrows in the other two panels show which action values were strengthened as a result of this path by one-step and n -step Sarsa methods. The one-step method strengthens only the last action of the sequence of actions that led to the high reward, whereas the n -step method strengthens the last n actions of the sequence, so that much more is learned from the one episode.

¿Puedo combinar distintos valores de n ?

- Podemos promediar retornos a distinto número de pasos.
- Por ejemplo, a 2 y 4 pasos:

$$\frac{1}{2}G_{t:t+2} + \frac{1}{2}G_{t:t+4}$$

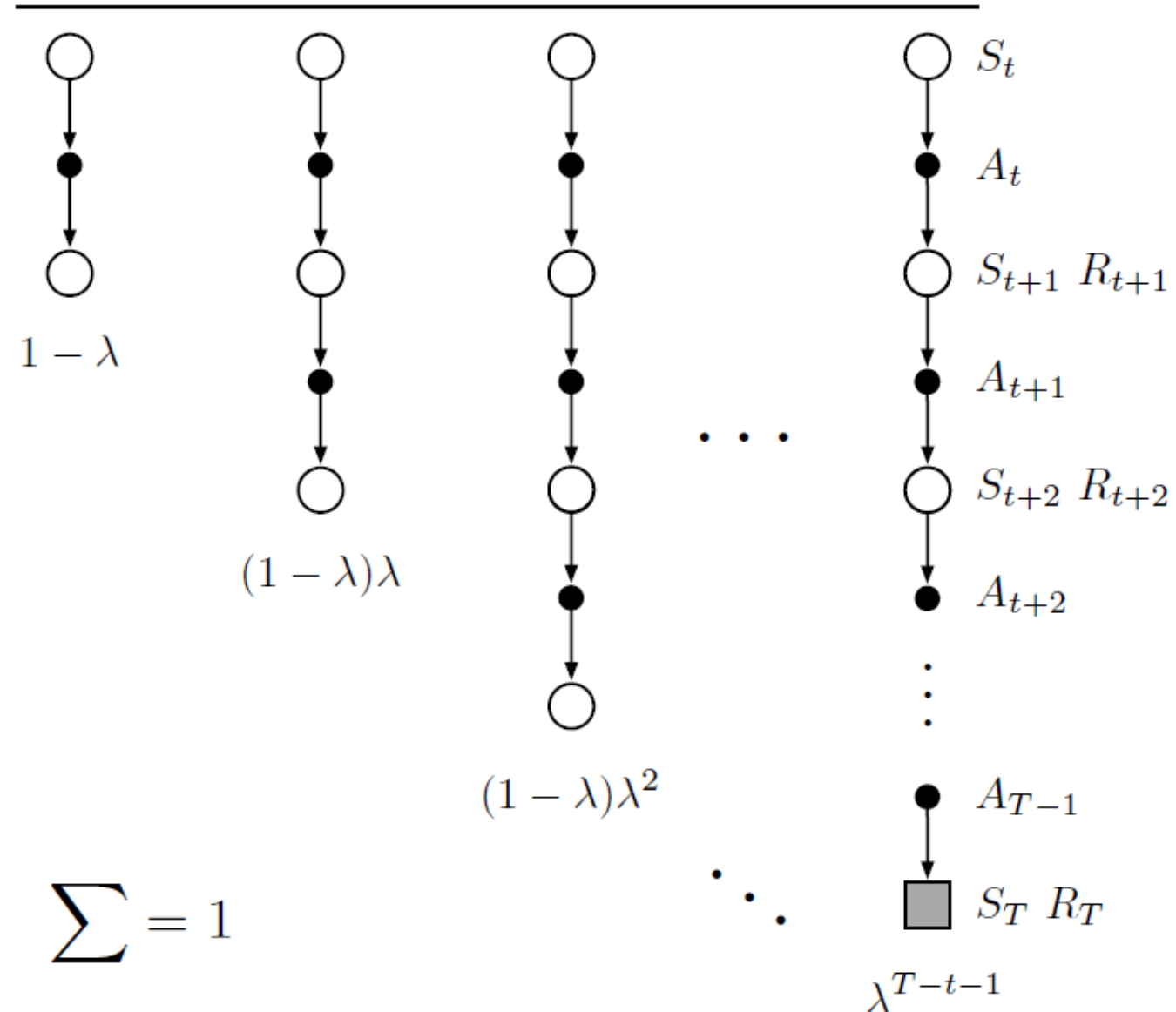
- De hecho, hay una estrategia para combinar Información de todos los retornos posibles.




$$\text{TD}(\lambda)$$

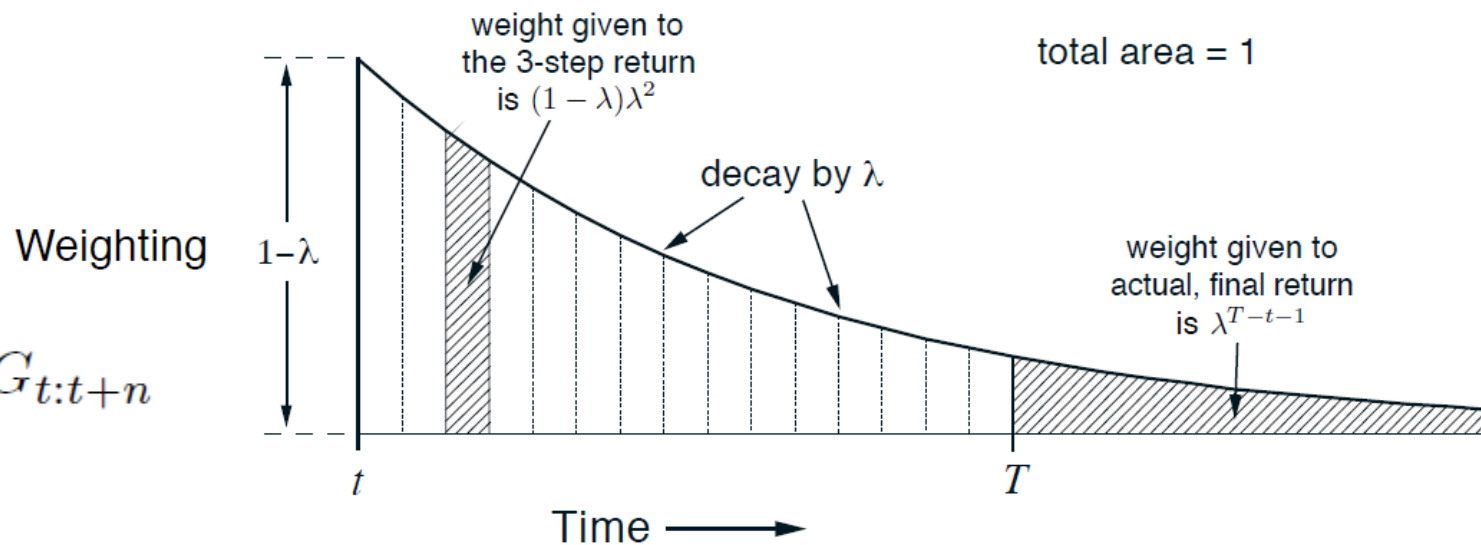
- Que puede usarse como el objetivo en nuestra regla de actualización:

$$\Sigma = 1$$



La función de ponderación

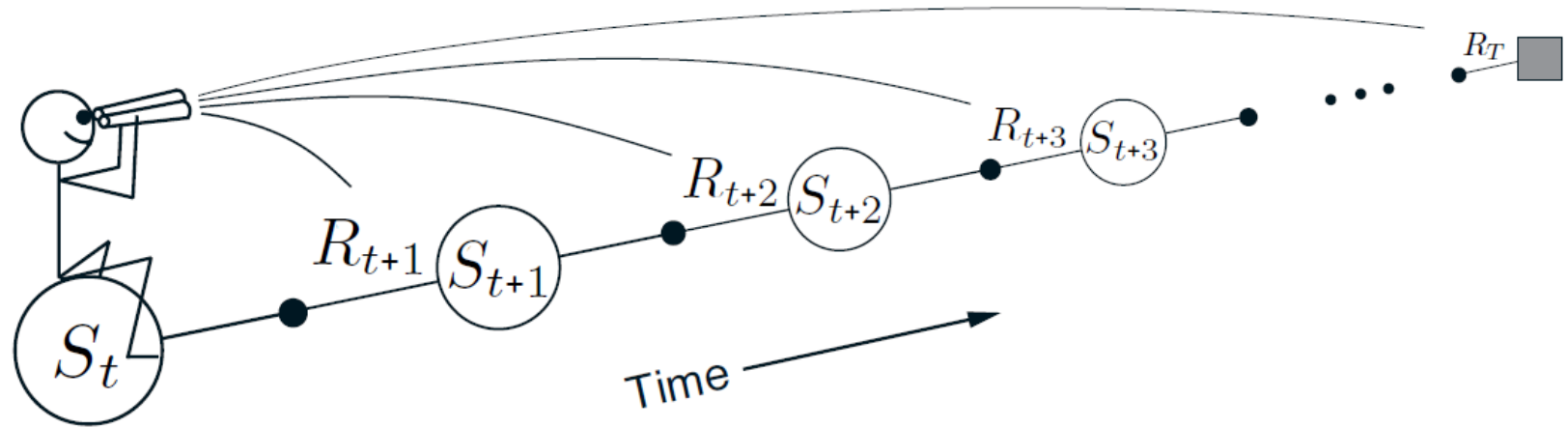
$$G_t^\lambda \doteq (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}$$



- El terminal es un estado “absorbent” $\rightarrow$ se le asigna todo el peso a partir de T.
- Se puede descomponer la fórmula de arriba:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t:t+n} + \lambda^{T-t-1} G_t$$

Interpretación hacia adelante de TD(λ)



- Esta aproximación de TD(λ) proporciona la base teórica para la combinación de distintos retornos.
- Desde cada estado, miro al futuro, actualizo hacia el retorno λ y ya no vuelvo más a “tocarlo”.
- Se trata de un algoritmo *off-line*.
comillas.edu

Interpretación hacia atrás $TD(\lambda)$

- La mirada adelante nos ha provisto del fundamento teórico. Sin embargo, en la práctica, tener que esperar a que termine un episodio tiene desventajas/limitaciones.
- La interpretación hacia atrás de $TD(\lambda)$ nos va a proveer de mecanismos para aprovechar la experiencia en el momento en el que se obtiene, de forma más eficiente desde el punto de vista computacional.
- Son formulas equivalentes (demostración disponible en S&B).
- Vamos a usar unas variables especiales conocidas como trazas de elegibilidad (*eligibility traces*).

Trazas de elegibilidad



- Ejemplo de David Silver: ¿Qué evento ha desencadenado la descarga eléctrica?
- Heurística basada en frecuencia → el timbre, porque ha sucedido más veces
- Heurística basada en recencia → la luz, porque ha sucedido hace poco
- Las trazas de elegibilidad combinan ambas heurísticas

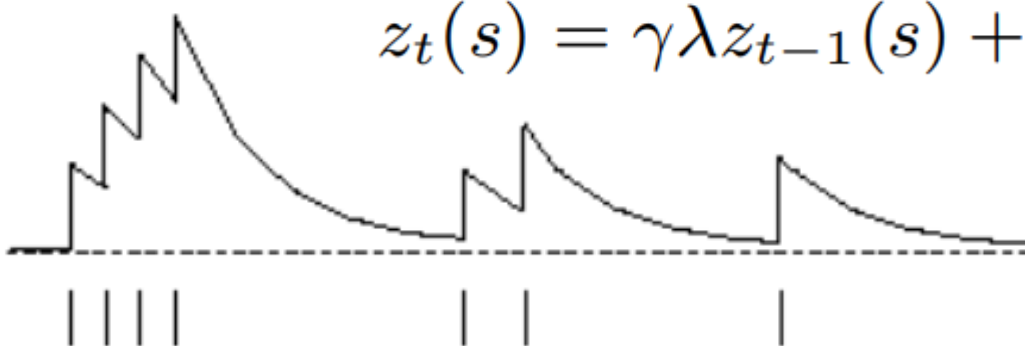
$$z_0(s) = 0$$

$$z_t(s) = \gamma \lambda z_{t-1}(s) + \mathbf{1}(S_t = s)$$

accumulating eligibility trace

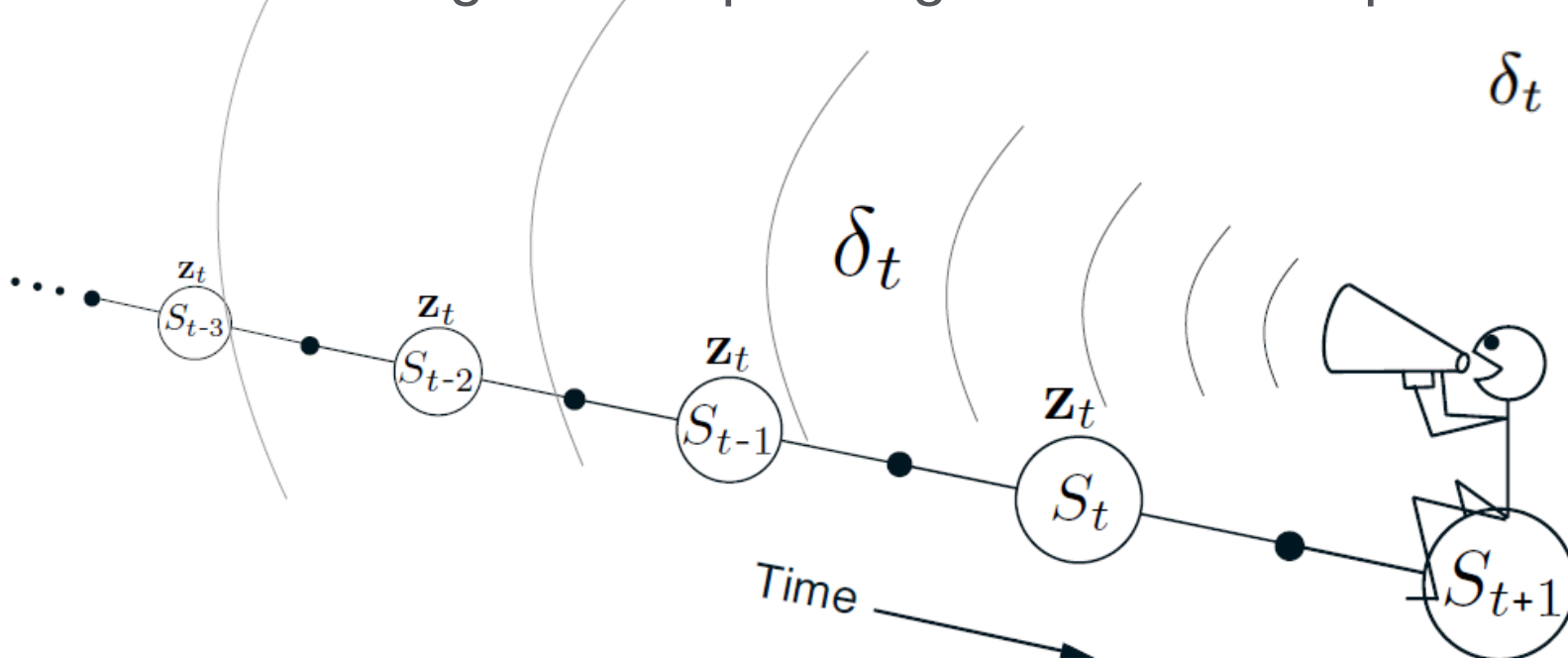
times of visits to a state

comillas



TD(λ) hacia atrás

- Cada vez que obtengo experiencia, calculo el error TD y lo propago a todos los estados.
- Actualizo los valores de TODOS los estados de forma proporcional a este error TD y a la traza de elegibilidad que tengo almacenada para cada uno de ellos.



$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

$$V(s) \leftarrow V(s) + \alpha \delta_t z_t(s)$$

TD(λ): de tabular a aproximación de función

- Recordemos la actualización de las trazas de elegibilidad que acabamos de ver:

$$z_t(s) = \gamma \lambda z_{t-1}(s) + \mathbf{1}(S_t = s)$$

- Ese 1 no es un valor arbitrario: se trata del **gradiente** de la función de valor para el caso de aproximación lineal con codificación *one-hot* de los estados (conurrencia entre caso tabular y aproximación lineal).

TD(λ): de tabular a aproximación de función

- En general, las trazas se actualizarán siguiendo esta expresión:

$$z_t \doteq \gamma \lambda z_{t-1} + \nabla \hat{v}(S_t, w_t)$$

- Y los pesos, por tanto, así:

$$w_{t+1} = w_t + \alpha \delta_t z_t$$

- Con esta aproximación, mediante el parámetro λ , controlamos un cierto compromiso entre sesgo y varianza, y podemos acelerar el aprendizaje, especialmente en las etapas iniciales.
- Observa que la actualización de los pesos depende del error TD, y este a su vez del objetivo TD, que se obtendrá de una u otra manera en función del algoritmo que usemos (Sarsa, Q-learning, etc.).